CSC490 Assignment A3

By Maria Ma: 1009054924, Zhixiao Fu: 1009834342,

Siyi Zhu: 1008793076, Chloe Yang: 1009261433

CSC490 · A3 PART 1 · OCT 13, 2025 INITIAL COMMIT

nanochat Model Architecture (Oct 13, 2025)

Input Token IDs

shape: (B, T) — integer token indices, T ≤ sequence_len=1024

EMBEDDING

Token Embedding — wte

nn.Embedding(vocab_size=50304, n_embd=768) → shape (B, T, 768)

⚠ No positional embedding added here — positions encoded via RoPE in attention.
⚠ Untied from lm_head (separate weight matrices, no weight tying).
⚠ Cast to bfloat16 immediately after init. Normal(0, 1.0) initialization.

NORM

RMSNorm (post-embedding)

F.rms_norm(x, (x.size(-1),)) — no learnable scale or bias

Applied immediately after embedding lookup, before any transformer blocks.

TRANSFORMER BLOCK × n_layer × n_layer (default 12)

RMSNorm (pre-attention)

F.rms_norm — no learnable params

ATTENTION

CausalSelfAttention — MQA-ready

Multi-Query Attention support · no bias · projections output (B, T, C), reshaped then transposed to (B, H, T, D) for SDPA

QKV PROJECTIONS — NO BIAS

Q	K	V
Linear(C → n_head·d) n_head=6	Linear(C → n_kv_head·d) n_kv_head=6	Linear(C → n_kv_head·d) n_kv_head=6

K, V can use fewer heads than Q (MQA). K/V heads repeated via repeat_kv() before dot-product.
When n_kv_head = n_head (default) = standard multi-head attention.

ROTARY POSITIONAL EMBEDDINGS (ROPE)

apply_rotary_emb(Q, cos, sin) | apply_rotary_emb(K, cos, sin)

cos/sin precomputed at init for seq_len × 10. Base θ = 10,000. Stored as bfloat16 buffers.
Encodes relative token positions by rotating Q and K in pairs of embedding dimensions.

QK NORM (APPLIED AFTER ROPE)

DEFAULT CONFIG (OCT 13)

sequence_len	1024
vocab_size	50,304 (GPT-2)
n_layer	12
n_head (Q)	6
n_kv_head (K/V)	6
n_embd	768
head_dim	128
MLP hidden	3072 (4×)
attention	MQA / SDPA
positional enc.	RoPE (θ=10k)

KEY DESIGN DECISIONS

- **RoPE**
  No learned positional embeddings. Relative positions encoded by rotating Q & K. Base θ=10,000.
- **QK Norm**
  RMSNorm on Q and K after RoPE. Prevents logit scale blow-up at large d_head.
- **MQA support**
  K/V head count can be < Q head count. Default equal (MHA). repeat_kv() expands for GQA.
- **ReLU²**
  relu(x).square() — sparser activations vs GELU. Computationally cheaper.
- **Pre-Norm**
  RMSNorm before each sub-layer (not after). Norm has no learnable params. Post-embedding norm too.
- **Untied weights**
  wte ≠ lm_head. Both separate matrices.
- **No bias anywhere**
  All nn.Linear layers: bias=False.
- **Logit softcap**
  15·tanh(x/15) bounds logits to ±15.

OPTIMIZER (OCT 13)

- **Muon**
  For all transformer block linear layers (matrix params). Momentum 0.95. Separate

norm(Q) | norm(K)

RMSNorm on Q and K after RoPE. Prevents attention logit explosion at large model sizes.

TRANSPOSE → (B, H, T, D) THEN REPEAT K/V FOR MQA

q.transpose(1,2), k.transpose(1,2), v.transpose(1,2)
repeat_kv(k, nrep) and repeat_kv(v, nrep) where nrep = n_head // n_kv_head

SCALED DOT-PRODUCT ATTENTION

F.scaled_dot_product_attention(q, k, v, is_causal=True)
PyTorch SDPA — causal mask applied. 3 code paths: training / inference full / inference chunked.

OUTPUT PROJECTION

c_proj — Linear(n_embd, n_embd, bias=False)
Initialized to zeros. Transpose back → (B, T, C)

+ Residual Add — x = x + attn_out

RMSNorm (pre-MLP)
F.rms_norm — no learnable params

MLP

Feed-Forward Network
4× expansion ratio · ReLU² activation · no bias

c_fc	ReLU²	c_proj
Linear(C → 4C, no bias)	F.relu(x).square()	Linear(4C → C, no bias)

c_proj initialized to zeros. c_fc initialized with custom std = 1/√fan_in · min(1, √(fan_out/fan_in)).

+ Residual Add — x = x + mlp_out

NORM

Final RMSNorm (post-blocks)
F.rms_norm — no learnable params

OUTPUT

LM Head — lm_head
Linear(n_embd=768, vocab_size=50304, bias=False) — untied from wte

Initialized to zeros. Separate from wte (no weight tying).

SOFTCAP

Logit Softcap
15 · tanh(logits / 15) — bounds logits to (-15, +15)

object.

- **AdamW**
  For wte + lm_head. β=(0.8, 0.95). LR ∝ 1/√d_model. Separate object. Two param groups.

DATA TYPES

wte weights	bfloat16
RoPE cos/sin	bfloat16
logits	float32
activations	bfloat16

Prevents extreme logit values from destabilizing gradients. Softcap runs in bfloat16; float32 cast happens after, before cross-entropy.

Cross-Entropy Loss

Training mode — targets provided

Raw Logits

Inference mode — no targets

nanochat Model Architecture (Feb 2026)

Input Token IDs

shape: (B, T) — integer token indices, T ≤ sequence_len=2048

CHANGED · EMBEDDING

Token Embedding — `wte`

nn.Embedding(padded_vocab_size, n_embd=768) — vocab padded to nearest ×64

vocab_size=32768 (nanochat BPE tokenizer, not GPT-2).
Padded to nearest multiple of 64 for tensor core efficiency.
lm_head slices back to true vocab_size.
Cast to bfloat16 on CUDA. Normal(0, 1.0) initialization. Still untied from lm_head.

NORM

RMSNorm (post-embedding)

F.rms_norm — no learnable params — unchanged

NEW

Save x_0 = norm(embedding)

Initial normalized embedding saved for use as skip connection in every block

x0 = x after post-embedding norm. Used in the $x0_\lambda \cdot x_0$ term in every block's residual update.

TRANSFORMER BLOCK × n_layer × n_layer (default 12)

NEW

Scaled Residual + x_0 Skip Connection

$x = resid_\lambda[i] \cdot x + x0_\lambda[i] \cdot x_0$

resid_λ: learnable per-layer scalar on the residual stream, init 1.0 (neutral).
x0_λ: learnable per-layer scalar blending initial embedding back in, init 0.1.
Both trained with AdamW. Inspired by modded-nanogpt / skip connections literature.

NEW — alternating layers

Value Embedding Lookup — `value_embeds[i](idx)`

nn.Embedding(vocab_size, n_kv_head × head_dim) — ResFormer-style

Present on layers where layer_idx % 2 == (n_layer - 1) % 2 (alternating, last layer always included).
Provides a direct token-level shortcut into the value computation, bypassing attention.
Initialized Uniform(±√3/√d) same as c_v. Cast to bfloat16.

RMSNorm (pre-attention)

CONFIG (FEB 2026 VS OCT 13)

sequence_len	2048 ↑ (was 1024)
vocab_size	32768 (was 50304)
n_layer	12 (same)
n_head (Q)	6 (same)
n_kv_head	6 (same)
n_embd	768 (same)
window_pattern	"SSSL" (new)
attention backend	FA3 (new)
value embeddings	ResFormer (new)
resid_λ / x0_λ	learnable (new)

NEW COMPONENTS

- **x_0 skip + resid_λ**
  Initial embedding re-injected at every block via learnable x0_λ scalar. Per-layer resid_λ scales residual stream. Prevents representational collapse.
- **Value Embeddings**
  ResFormer: token embedding shortcut → V via input-dependent gate. Alternating layers only. Bypasses attention bottleneck for token identity.
- **Flash Attention 3**
  Native FA3 on H100. (B,T,H,D) layout eliminates transpose. SDPA fallback on other hardware.
- **Sliding Window Attn**
  Per-layer window pattern "SSSL". 3/4 of layers use half-context window. Reduces FLOPs at 2048 context.
- **Unified MuonAdamW**
  Single optimizer object, 5+ param groups with separate LR/beta per group type.

UNCHANGED FROM OCT 13

- **RoPE (θ=10000)**
  Same formula, same base. bfloat16 buffers.
- **QK Norm**
  RMSNorm on Q, K after RoPE.
- **ReLU² in MLP**
  relu(x).square() — unchanged.
- **Pre-norm RMSNorm**

F.rms_norm — no learnable params — unchanged

ATTENTION

CausalSelfAttention — GQA + FA3 + Sliding Window

Grouped Query Attention · Flash Attention 3 · native (B,T,H,D) layout · no bias

QKV PROJECTIONS — NATIVE FA3 LAYOUT (B, T, H, D) — NO TRANSPOSE

Q	K	V
Linear(C → n_head·d)	Linear(C → n_kv_head·d)	Linear(C → n_kv_head·d)

Shape is (B, T, H, D) throughout — no transpose needed for FA3, unlike Oct 13.

VALUE RESIDUAL INJECTION (RESFORMER) — NEW ✦

V = V + gate · ve

gate = 2 · σ(Linear(x[..., :32])) ∈ (0, 2) per KV head

ve = value_embeds[i](idx) reshaped to (B, T, n_kv_head, head_dim).
gate uses only first 32 dims of x (ve_gate_channels=32) to compute per-head gate.
gate_init=zeros → sigmoid(0)=0.5 × 2 = 1.0 neutral at init.
Only runs on layers that have value_embeds (alternating).

ROPE ON Q AND K

apply_rotary_emb(Q, cos, sin) | apply_rotary_emb(K, cos, sin)

Same RoPE formula as Oct 13. Base θ=10,000. bfloat16 cos/sin buffers.

QK NORM

norm(Q) | norm(K)

FLASH ATTENTION 3 — NEW ✦

flash_attn.flash_attn_func(q, k, v, causal=True, window_size=...)

FA3 on Hopper+ (H100); PyTorch SDPA fallback elsewhere

Input shape (B,T,H,D) — no transpose needed.
window_size=(left, right) controls the attention span.

PER-LAYER SLIDING WINDOW ATTENTION — NEW ✦

S layers — 1024
L layers — full context = 2048
last layer — always full

Default pattern "SSSL": 3 short-window layers then 1 full-context layer, tiled across all n_layer.
Short window = seq_len // 2. Long window = seq_len. Final layer always full.
Reduces attention FLOPs on 3/4 of layers. Enables practical 2048 context length.

OUTPUT PROJECTION

No learnable params. Pre-attn, pre-MLP, post-embed, final.

- No bias in Linear
  All nn.Linear: bias=False.
- Logit softcap 15
  15·tanh(x/15).
- Untied wte / lm_head
- Muon for matrix params

c_proj — Linear(n_embd, n_embd, bias=False) — initialized to zeros

+ Residual Add — x = x + attn_out

RMSNorm (pre-MLP)

F.rms_norm — no learnable params — unchanged

MLP

Feed-Forward Network — unchanged from Oct 13

4× expansion · ReLU² activation · no bias

c_fc	ReLU²	c_proj
Linear(C → 4C)	F.relu(x).square()	Linear(4C → C)

+ Residual Add — x = x + mlp_out

NORM

Final RMSNorm

F.rms_norm — no learnable params

OUTPUT

LM Head — `lm_head`

Linear(n_embd, padded_vocab_size) → sliced to [:vocab_size]

Output padded to ×64 multiple, sliced back: logits[..., :self.config.vocab_size].
Initialized Normal(0, 0.001) — small but nonzero (changed from Oct 13 zeros).

SOFTCAP

Logit Softcap — unchanged

logits.float() → 15 · tanh(logits / 15)

Same softcap=15. Order changed: Oct 13 ran tanh in bfloat16 then cast to float32; Feb 2026 casts to float32 first, then applies tanh.

Cross-Entropy Loss

Training — targets provided

Raw Logits

Inference — no targets

Three Proposed New Architecture Changes

Context

The Feb 2026 nanochat already incorporates many modern improvements: RoPE, QK Norm, GQA, sliding window attention, Flash Attention 3, value embeddings, and learnable residual scalars. The three changes below address components that remain absent yet have strong empirical support in recent literature.

1 Higher RoPE Base Frequency (θ = 500,000)

1-line change · Zero new parameters · Acknowledged TODO in nanochat source

Su et al., 2021 · arXiv:2104.09864
Meta AI, 2024 · arXiv:2407.21783 (LLaMA 3)

MOTIVATION

The current nanochat uses RoPE base θ=10,000, kept from GPT-NeoX despite nanochat's Feb 2026 version having doubled context to 2048. Low θ causes high-frequency rotary dimensions to complete multiple full rotations within the window, making positional signals ambiguous at longer sequences. Nanochat's own source has a TODO: *"bump base theta more, e.g. 100K is more common more recently."* LLaMA 3 uses θ=500K.

TECHNICAL DETAILS

Single constant change in `_precompute_rotary_embeddings`:

```
base=500000 # was 10000
```

Higher θ → lower inv_freq = 1/(θ^(2i/d)) → slower rotation per token → each frequency band encodes a wider positional range before aliasing. No weight changes, no new parameters, no optimizer changes.

WHY NOT ALREADY HERE

The original θ=10,000 was inherited from nanoGPT for simplicity. The nanochat codebase itself acknowledges this as a known gap. LLaMA 3 validated θ=500K for long-context performance; the choice has since become standard across Mistral, Qwen, and Gemma families.

EXPECTED IMPACT

Minimal for short sequences (≤512 tokens). Meaningful at 2048 tokens: reduced positional aliasing, more distinct per-position encodings. Enables future context extension (4K+) from the same checkpoint via YaRN/linear interpolation. Zero parameter and FLOP overhead.

2 Learnable RMSNorm Scale γ

·10-line module · 3,072 new params (<0.1%) · Universal in all peer LLMs

Zhang & Sennrich, 2019 · arXiv:1910.07467
Warner et al., 2025 · arXiv:2412.13663 (ModernBERT, ACL 2025)

MOTIVATION

TECHNICAL DETAILS

WHY NOT ALREADY HERE

EXPECTED IMPACT

Nanochat uses purely functional `F.rms_norm(x, (C,))` with no learnable parameters — every channel is normalized to equal scale. Every production LLM that uses RMSNorm (LLaMA 1/2/3, Gemma, Mistral, Qwen, ModernBERT) includes a learnable per-channel scale $\gamma \in \mathbb{R}^d$. Without it, all embedding dimensions are on equal footing after normalization, limiting the model's representational flexibility.

```
class RMSNorm(nn.Module):
  def __init__(self, d):
    super().__init__()
    self.weight =
nn.Parameter(torch.ones(d))
  def forward(self, x):
    return F.rms_norm(x,
(x.size(-1),),
self.weight)
```

Replace all 4 functional `norm(x)` calls in `Block` and `GPT.forward` with instances of this module. Initialize γ=1.0 (identity). Add γ parameters to AdamW group.

Karpathy explicitly chose parameter-free RMSNorm to minimize model complexity. This is a deliberate minimalist decision, not an oversight. However, the learnable γ is considered necessary by essentially every other LLM codebase — its absence is the outlier position.

Small but consistent: ·0.1–0.3% perplexity improvement. Allows the model to break post-normalization symmetry, assigning different representational weights to different hidden dimensions. Near-zero parameter and FLOP overhead. Low risk of destabilizing training.

3 SwiGLU Activation in MLP

·20 lines · Extra weight matrix · Highest perplexity impact of the three changes

Shazeer, 2020 · arXiv:2002.05202
Touvron et al., 2023 · arXiv:2307.09288 (LLaMA 2)

MOTIVATION

Nanochat's MLP uses ReLU², which hard-zeros negative activations. GLU variants (SwiGLU, GeGLU) have consistently outperformed ReLU-family activations across modern LLMs. LLaMA 1/2/3, PaLM, Gemma, Mistral, and ModernBERT all use SwiGLU or GeGLU. The gating mechanism allows multiplicative suppression or amplification per hidden dimension, rather than binary zeroing.

TECHNICAL DETAILS

Replace 2-matrix MLP with 3-matrix gated version:

```
gate = Linear(C→H,
bias=False)
up = Linear(C→H,
bias=False)
down = Linear(H→C,
bias=False)
# forward:
x =
down(F.silu(gate(x)) *
up(x))
```

Hidden dim $H = \lceil (2/3 \times 4C) / 64 \rceil \times 64 = 2048$ when C=768. The 2/3 reduction factor is from Shazeer (2020) Table 1: to keep FLOPs iso-parametric, the three-matrix GLU layer uses hidden dim = (2/3) × original d_ff. For nanochat: (2/3) × 3072 = 2048, rounded to nearest multiple of 64.

WHY NOT ALREADY HERE

ReLU² was competitive at time of initial design (comparable to squared ReLU in PaLM). SwiGLU requires a third weight matrix and more complex forward pass. The 2/3-adjusted hidden dim formulation makes it a principled iso-parametric swap rather than a parameter increase.

EXPECTED IMPACT

Shazeer (2020) Table 1: SwiGLU achieves log-perplexity 1.636 vs ReLU baseline 1.677 at equal parameters and FLOPs — roughly a 2.4% improvement in log-perplexity. LLaMA 2 (Touvron et al. 2023) uses SwiGLU throughout and reports faster convergence vs ReLU baselines. Improved gradient flow through the gate path. Small downside: 3 weight matrices instead of 2 → ·50% more MLP memory.

References

Su, J., et al. (2021). RoFormer: Enhanced transformer with rotary position embedding. *arXiv:2104.09864*.

Meta AI (2024). The Llama 3 herd of models. *arXiv:2407.21783*.

Zhang, B., & Sennrich, R. (2019). Root mean square layer normalization. *NeurIPS 2019 / arXiv:1910.07467*.

Warner, B., et al. (2025). Smarter, better, faster, longer. *ACL 2025 / arXiv:2412.13663*.

Shazeer, N. (2020). GLU variants improve transformer. *arXiv:2002.05202*.

Touvron, H., et al. (2023). Llama 2. *arXiv:2307.09288*.

Karpathy, A. (2025). nanochat: A tiny chatbot arena and training harness. *github.com/karpathy/nanochat*.

Part 2 (change 3: SwiGLU Activation in MLP)

<u>Choose a smaller nanochat configuration (picochat), justify your choice and train a baseline model</u>

We use a smaller nanochat configuration called **picochat** as our baseline model. This configuration has **depth 12 (d12)** with approximately **125M parameters**, using **embedding dimension 768**, **head dimension 128**, and **aspect ratio 64**, which follow the default settings in the nanochat repository. The model is trained on **8 shards of the FineWeb-EDU dataset** with a **sequence length of 2048** and **device batch size of 16** on **8×H100 GPUs**.

```
## Base model training
timestamp: 2026-03-04 19:05:07

- run: picochat_baseline
- device_type:
- fp8: False
- fp8_recipe: tensorwise
- depth: 12
- aspect_ratio: 64
- head_dim: 128
- max_seq_len: 2048
- window_pattern: SSSL
- num_iterations: -1
- target_flops: -1.0000
- target_param_data_ratio: 10.5000
- device_batch_size: 16
- total_batch_size: -1
- embedding_lr: 0.3000
- unembedding_lr: 0.0040
- weight_decay: 0.2000
- matrix_lr: 0.0200
- scalar_lr: 0.5000
- adam_beta1: 0.8000
- adam_beta2: 0.9500
- warmup_ratio: 0.0000
- warmdown_ratio: 0.5000
- final_lr_frac: 0.0000
- resume_from_step: -1
- eval_every: 250
- eval_tokens: 20,971,520
- core_metric_every: 2000
- core_metric_max_per_task: 500
- sample_every: 2000
- save_every: 1000
- model_tag: None
- Number of parameters: 286,262,424
- Number of FLOPs per token: 8.021676e+08
- Calculated number of iterations: 2205
- Number of training tokens: 1,156,055,040
- Tokens : Scaling params ratio: 10.4999
- DDP world size: 8
- warmup_ratio: 0.0000
- warmdown_ratio: 0.5000
- final_lr_frac: 0.0000
- Minimum validation bpb: 0.9041
- Final validation bpb: 0.9041
- CORE metric estimate: 0.1230
- MFU %: 36.84%
- Total training flops: 9.273498e+17
- Total training time: 5.84m
- Peak memory usage: 16372.35MiB
```

We chose this configuration for three main reasons. First, it allows **fast iteration**. The d12 model is small enough to finish training in about **10 minutes**, which makes it practical to run multiple experiments such as the baseline and the SwiGLU variant. Second, it keeps the **computational cost low**, since a smaller model and limited dataset shards reduce GPU usage while still producing meaningful evaluation results. Third, it enables a **fair comparison** between model variants. In our experiments, the architecture depth, dataset, optimizer settings, and training schedule remain the same, so any performance difference can be attributed to the change in the MLP design rather than other factors.

We trained a **baseline picochat model with the original ReLU² MLP**, following the setup for a d12 model. After pretraining, we compute evaluation metrics.

The baseline training completed successfully with a total training time of approximately **5.84 minutes** as shown in figure above**.** The final evaluation results are shown in below figure:

- **Validation bpb:** 0.9041
- **CORE metric:** 0.1230

<u>Train a picochat with each change and compare the performance to the baseline picochat</u>

<u>Create a table summarizing the results and include commentary on the impact of each change</u>

To study the effect of activation functions in the transformer MLP, we train another picochat model where the baseline **ReLU² MLP** is replaced with **SwiGLU**, while keeping the rest of the model configuration the same. This allows us to isolate the impact of the MLP change on model performance.

```
## Base model training
timestamp: 2026-03-04 20:27:10

- run: picochat_swiglu
- device_type:
- fp8: False
- fp8_recipe: tensorwise
- depth: 12
- aspect_ratio: 64
- head_dim: 128
- max_seq_len: 2048
- window_pattern: SSSL
- num_iterations: -1
- target_flops: -1.0000
- target_param_data_ratio: 10.5000
- device_batch_size: 16
- total_batch_size: -1
- embedding_lr: 0.3000
- unembedding_lr: 0.0040
- weight_decay: 0.2000
- matrix_lr: 0.0200
- scalar_lr: 0.5000
- adam_beta1: 0.8000
- adam_beta2: 0.9500
- warmup_ratio: 0.0000
- warmdown_ratio: 0.5000
- final_lr_frac: 0.0000
- resume_from_step: -1
- eval_every: 250
- eval_tokens: 20,971,520
- core_metric_every: 2000
- core_metric_max_per_task: 500
- sample_every: 2000
- save_every: 1000
- model_tag: d12_swiglu
- Number of parameters: 286,262,424
- Number of FLOPs per token: 8.021676e+08
- Calculated number of iterations: 2205
- Number of training tokens: 1,156,055,040
- Tokens : Scaling params ratio: 10.4999
- DDP world size: 8
- warmup_ratio: 0.0000
- warmdown_ratio: 0.5000
- final_lr_frac: 0.0000
- Minimum validation bpb: 0.9064
- Final validation bpb: 0.9064
- CORE metric estimate: 0.1334
- MFU %: 38.89%
- Total training flops: 9.273498e+17
- Total training time: 5.14m
- Peak memory usage: 16307.96MiB
```

After training, we evaluate the SwiGLU model using the same evaluation pipeline as the baseline. The results are summarized in the table below.

Model	MLP Type	Validation bpb	CORE Metric	Training Time

Baseline picochat	ReLU²	**0.9041**	0.1230	5.84 min
picochat (SwiGLU)	SwiGLU	0.9064	**0.1334**	**5.14 min**

From the results, the baseline picochat performs slightly better than the SwiGLU variant on the validation bpb metric. The baseline achieves a lower validation bpb (0.9041 vs. 0.9064), indicating slightly stronger language modeling performance in this experiment. However, the SwiGLU variant obtains a higher CORE score (0.1334 vs. 0.1230), suggesting slightly better benchmark performance on the evaluation tasks.

The SwiGLU model trains slightly faster (5.14 minutes compared to 5.84 minutes for the baseline). However, this improvement in training speed does not translate into better validation bpb in this small scale setup.

One possible explanation is that the picochat model is relatively small, and the advantages of SwiGLU may become more noticeable in larger models or longer training runs.

Track them using weights and biases or a similar tool to visualize the training process and compare the different model

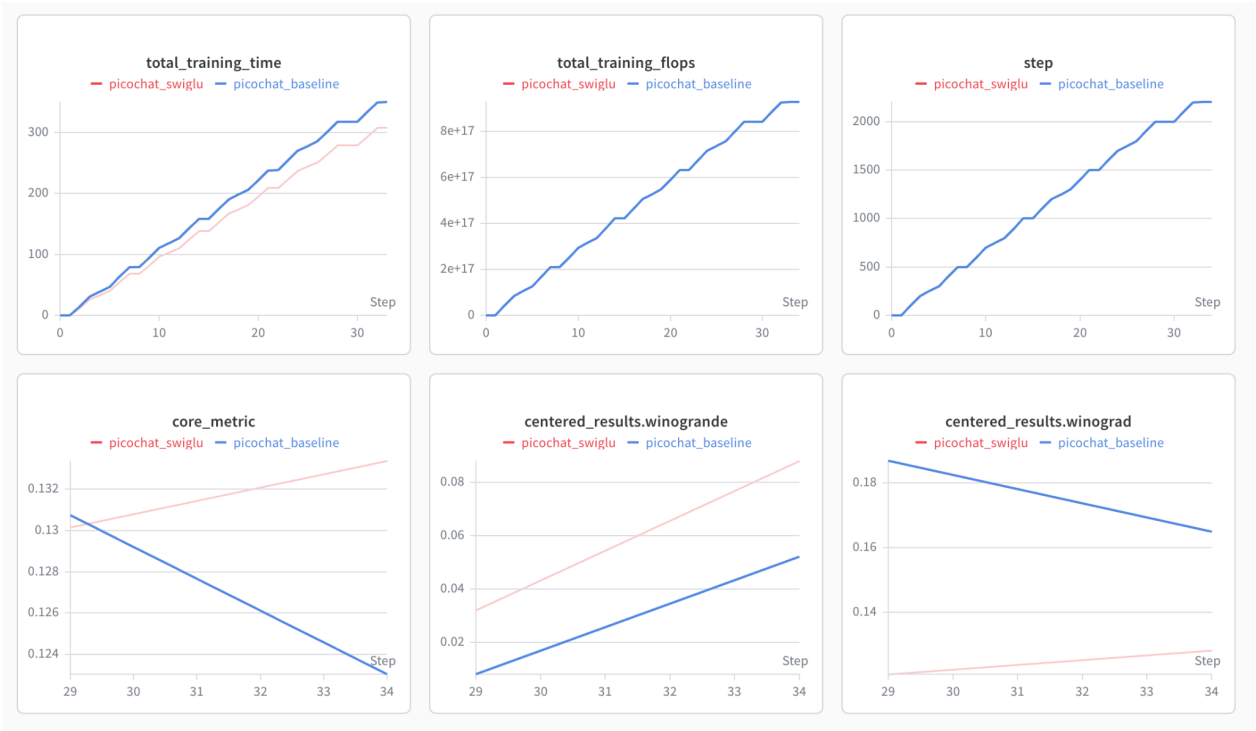
We track the training process and evaluation metrics using Weights & Biases (W&B). The figure below shows several metrics recorded during training for both the baseline picochat model and the SwiGLU variant.

The plots of total training time, training FLOPs, and training steps illustrate the compute progression during training. Both models follow a very similar training trajectory and reach the same number of training iterations, confirming that the comparison is performed under a consistent compute budget.

The CORE metric plot shows the evaluation performance during the later stage of training. The SwiGLU variant achieves a slightly higher CORE value than the baseline model, which is consistent with the quantitative results reported earlier.

We also visualized an example benchmark task, Winogrande, to illustrate how model performance differs across individual tasks. The results indicate that performance differences between the two models vary across benchmarks, but overall the differences remain relatively small.

These visualizations demonstrate how W&B can be used to monitor training dynamics and directly compare different model variants during experimentation.



Discussion of the cost of training these models

Since the experiments were conducted on GPU resources with limited credits, we carefully selected a small model configuration to keep the training cost manageable. The **picochat (d12)** model contains approximately **125M parameters** and is significantly smaller than larger model configurations used in the project. This allows the model to be trained quickly while still producing meaningful evaluation results.

Each training run processes approximately **1.16B tokens over 2205 iterations** and completes in about **5–6 minutes** on **8×H100 GPUs**. According to the training logs, the total compute usage is approximately **9.27×10^{17} training FLOPs** per run. Because the model is relatively small, this setup enables multiple experiments to be performed within a short time and within the available credit budget.

Using this configuration allowed us to train both the **baseline picochat model** and the **SwiGLU variant** while keeping the total compute cost low. Instead of using a larger model or more training tokens, we prioritized **fast iteration and controlled experiments**, which is particularly important when experimenting with architectural changes.

Overall, selecting the picochat configuration provides a good balance between **experimental efficiency and meaningful model comparison**, enabling us to complete the experiments without exceeding the available compute credits.

Comment how your ablations may translate for a larger run

The ablation experiment on the picochat configuration (d12) provides a small scale proxy for understanding how architectural changes may behave in larger models. In this experiment, replacing the baseline ReLU² MLP with SwiGLU does not improve validation bpb, as the baseline achieves a slightly lower value. However, the SwiGLU variant obtains a slightly higher CORE score.

When translating this result to larger runs, it suggests that the SwiGLU change does not provide a clear advantage at this small scale under the current training budget. However, the picochat model is relatively small and the training run is short compared to typical large scale language model training. As a result, the observed performance difference may not fully reflect the behavior of larger models.

In larger model configurations with greater capacity and longer training runs, architectural changes in the MLP may interact differently with training dynamics. Therefore, while the current ablation does not show a clear advantage for SwiGLU at the d12 scale, additional experiments on larger models would be needed to determine whether the effect changes when scaling up the model size or training duration.

Part 2 (change 2: Learnable RMSNorm Scale γ)

Choose a smaller nanochat configuration (picochat), justify your choice and train a baseline model

We use the same picochat (d12) baseline configuration as the SwiGLU ablation: depth 12, approximately 125M parameters, embedding dimension 768, head dimension 128, aspect ratio 64, trained on 8 shards of FineWeb-EDU, sequence length 2048, device batch size 16 on 8×H100 GPUs. The baseline results (ReLU², parameter-free RMSNorm) are shared with the SwiGLU experiment above.

Train a picochat with each change and compare the performance to the baseline picochat

Create a table summarizing the results and include commentary on the impact of each change

To study the effect of learnable RMSNorm scale (γ), we train another picochat model where the baseline parameter-free RMSNorm (F.rms_norm(x, (c,)), no learnable γ) is replaced with a learnable RMSNorm that adds a per-channel scale parameter γ initialized to ones (F.rms_norm(x, (c,), weight=γ)). The change is applied at all four normalization positions in each transformer block (post-embedding norm, pre-attention norm, pre-MLP norm, and post-block norm), totalling (1 + 3 × n_layer) × n_embd additional parameters. For the d12 model this adds 1 + 3 × 12 = 37 γ vectors of dimension 768, for a total of 28,416 additional parameters (~0.02% of model size). All other architecture, optimizer, and training settings remain unchanged.

The γ parameters are trained with AdamW at the same learning rate as the embedding parameters (embedding_lr × dmodel_lr_scale), as they play a similar role in scaling activations.

After training, we evaluate the learnable RMSNorm model using the same evaluation pipeline as the baseline. The results are summarized in the table below.

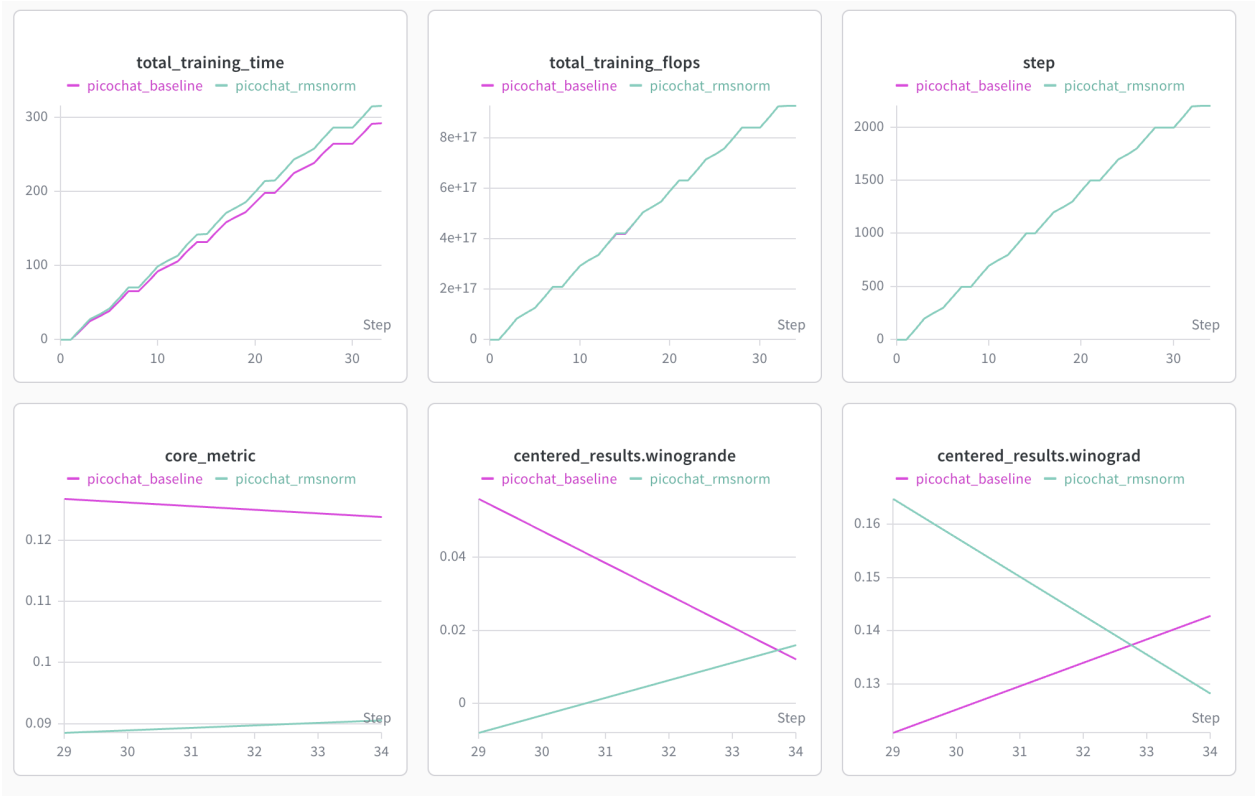
Model	Norm Type	Validation bpb	CORE Metric	Training Time
Baseline picochat	Parameter-free RMSNorm	0.9041	0.1230	5.84 min
picochat (SwiGLU)	Parameter-free RMSNorm	0.9064	0.1334	5.14 min
picochat (Learnable RMSNorm)	Learnable γ RMSNorm	0.9710	0.0906	5.26 min

The learnable RMSNorm variant performs worse than the baseline on both metrics: validation bpb is higher (0.9710 vs. 0.9041) and CORE metric is lower (0.0906 vs. 0.1230). Training time is similar to the SwiGLU variant at 5.26 minutes.

The degraded performance is likely due to several factors. First, the picochat model is very small (d12, 8 shards) and trains for a short time (2205 steps), which may not provide enough gradient signal for the γ parameters to converge to useful values. At initialization γ = 1, the model is equivalent to the baseline, but the optimizer must learn to adjust these parameters away from the neutral initialization. Second, adding γ changes the effective learning rate of the normalization layers in a way that may interact poorly with the Muon optimizer used for the weight matrices, since the γ parameters follow a different update rule. Third, the relatively high learning rate inherited from the embedding group may cause the γ parameters to overshoot early in training before the residual stream has stabilized.

Track them using weights and biases or a similar tool to visualize the training process and compare the different models

The training process is tracked using Weights & Biases (W&B).



W&B comparison: picochat_baseline vs picochat_rmsnorm

The figure above compares picochat_baseline (pink) and picochat_rmsnorm (green) across several W&B metrics. The top row shows total training time, total training FLOPs, and step count — both models follow nearly identical compute trajectories, confirming the comparison is made under the same compute budget. The bottom row shows the CORE metric and two individual benchmark tasks (Winogrande and Winograd). The baseline consistently achieves a higher CORE metric (~0.123) compared to the RMSNorm variant (~0.091), consistent with the quantitative results reported in the table above. Performance on individual tasks varies: on Winogrande the baseline edges ahead, while on Winograd the two models cross over, illustrating that the difference is not uniform across tasks.

<u>Discussion of the cost of training these models</u>

The learnable RMSNorm experiment adds only 28,416 parameters (~0.02%) to the baseline d12 model, making it effectively zero-cost from a parameter and FLOPs perspective. The total training compute is approximately the same as the baseline and SwiGLU runs: 9.27×10^{17} training FLOPs over 2205 iterations in about 5.26 minutes on 8×H100 GPUs. The γ parameters are so small relative to the total model size that they do not measurably affect per-step compute.

The additional cost of this experiment compared to the baseline is therefore essentially just one training run of 5.26 minutes, identical in cost to the SwiGLU ablation. Using the same picochat d12 setup allows direct comparison across all three configurations (baseline, SwiGLU, learnable RMSNorm) under the same compute budget.

Comment how your ablations may translate for a larger run

At the picochat (d12) scale, the learnable RMSNorm variant does not improve over the baseline. However, the literature (LLaMA 1/2/3, Gemma, Mistral, ModernBERT) consistently shows that learnable γ provides small but consistent improvements in larger models trained for longer. The most likely explanation is that the benefit of per-channel rescaling requires sufficient model capacity and training data for the γ parameters to learn meaningful non-uniform scalings across embedding dimensions.

In a larger run (e.g. d20 or d24 trained on 240+ shards), the γ parameters would have much more gradient signal and could potentially learn to amplify informative dimensions and suppress noise dimensions post-normalization. The near-zero cost in parameters and FLOPs also means the change scales freely to larger models. Based on the literature citing 0.1–0.3% perplexity improvements in larger settings, it is reasonable to expect that the learnable RMSNorm change would show a modest positive effect at scale even though it does not help at the picochat level.

Part 3

Training:

Training and storing the checkpoints with 512 and 2048 context window, each with 2000 steps and 4000 steps(first 2000 steps based on the 512 context window)

Evaluation of 2 checkpoints:

Justification of modifying the CORE metric code:

a. `scripts/base_eval.py`
   i. `evaluate_core(..., max_seq_len=sequence_len)`
   ii. `sequence_len` comes from the loaded checkpoint's meta (512 for step 2000, 2048 for step 4000).
   iii. That value is passed into CORE so prompts are truncated to the model's context length.
b. `nanochat/core_eval.py`
   i. `evaluate_example(..., max_seq_len=None)` and `evaluate_task(..., max_seq_len=None)`
   ii. Truncation now uses, in order: the passed-in `max_seq_len`, or `model.max_seq_len` if present. So both nanochat (via argument) and HuggingFace (via attribute) are handled.
c. With this, checkpoint 1 (step 2000, 512 context) gets CORE prompts truncated to 512 tokens and no longer hits the rotary cache assert.

Result from the comparison of 2 checkpoints:

Csv file link: [EyeHearU/a3/part3/base_eval at freya-a3-part3 · UofT-CSC490-W2026/EyeHearU]

Setup

Checkpoint	Description	Step	Context length	Training
Checkpoint 1	`base_model_002000`	2,000	512	Trained from scratch on a portion of FineWeb-EDU at 512 tokens
Checkpoint 2	`base_model_004000`	4,000	2048	Resumed from Checkpoint 1; continued training for 2,000 steps at 2048 tokens

Both models are picochat (depth 6, ~11M params). Evaluation uses the CORE benchmark (DCLM-style) with 100 examples per task and bits-per-byte (BPB) on the validation split.

Aggregate Results

Metric	Checkpoint 1 (step 2k, ctx 512)	Checkpoint 2 (step 4k, ctx 2048)
CORE (centered)	0.0453	0.0386
Val BPB (from training logs)	1.264	1.183

- **CORE**: Checkpoint 1 is slightly higher (0.0453 vs 0.0386). The 2048-continuation did not improve the aggregate CORE score on this benchmark.
- **Val BPB**: Checkpoint 2 has lower validation BPB (1.183 vs 1.264), indicating better next-token prediction on the pretraining distribution after the 2048 phase.

Task-Level Comparison (Accuracy)

Tasks where **Checkpoint 2 (2048)** does better:

Task	Ckpt 1 (512)	Ckpt 2 (2048)	Δ
hellaswag_zeroshot	0.26	0.27	+0.01
copa	0.50	0.51	+0.01
commonsense_qa	0.29	0.33	+0.04
piqa	0.45	0.47	+0.02
hellaswag	0.27	0.28	+0.01
winograd	0.45	0.49	+0.04
winogrande	0.45	0.49	+0.04
bigbench_dyck_languages	0.00	0.02	+0.02
bigbench_cs_algorithms	0.13	**0.41**	**+0.28**
bigbench_operators	0.12	0.15	+0.03

coqa	0.00	0.01	+0.01

Tasks where **Checkpoint 1 (512)** does better:

Task	Ckpt 1 (512)	Ckpt 2 (2048)	Δ
openbook_qa	0.25	0.24	−0.01
lambada_openai	0.20	0.18	−0.02
agi_eval_lsat_ar	0.28	0.19	−0.09
boolq	**0.75**	0.52	**−0.23**
bigbench_language_identification	0.26	0.23	−0.03

No change: arc_easy (0.34), arc_challenge (0.21), and several tasks at 0.00 for both.

Summary and Interpretation

1. **CORE vs BPB**
   Checkpoint 2 has better validation BPB but slightly lower CORE. So extending context and continuing training helped language modeling (BPB) more than it helped this mix of in-context evaluation tasks on average.
2. **Where 2048 helps**
   The 2048 checkpoint improves on several tasks that can benefit from longer context (e.g. commonsense_qa, winograd, winogrande) and shows a large gain on **bigbench_cs_algorithms** (+0.28). That suggests the longer-context phase helped on some reasoning and multi-turn style tasks.
3. **Where 512 wins**
   The 512 checkpoint is much stronger on **boolq** (0.75 vs 0.52) and better on **agi_eval_lsat_ar** and **lambada_openai**. Possible factors: different effective capacity

allocation after continuation, or CORE prompt length / truncation (e.g. 512 ckpt sees truncated prompts, which might sometimes help on these formats).

4. **Conclusion**
   Continuing from 512 to 2048 context improved validation BPB and several CORE tasks (especially bigbench_cs_algorithms, winograd, winogrande, commonsense_qa), but hurt a few others (notably boolq), so the overall CORE score was slightly lower. For a write-up, you can summarize: the 2048 continuation improves language modeling and some longer-context tasks, while the 512 checkpoint remains stronger on a few specific benchmarks (boolq, agi_eval_lsat_ar).

Part 4: Final nanochat

Configuration for the final nanochat model:

We trained `d20_swiglu` using the Modal speedrun pipeline (`nanochat_modal.py::stage_pretrain_swiglu`) and the full pretraining token budget computed by the nanochat scripts for depth 20.

- **architecture change**: ReLU² MLP → **SwiGLU MLP**
- **depth**: 20 (nanochat scale for this project)
- **DDP world size**: 8 GPUs
- **device batch size**: 16 sequences / GPU
- **max seq len**: 2048
- **window pattern**: SSSL

Justification of configuration: architecture and scale

<u>Why SwiGLU (and why we keep parameter-free RMSNorm)</u>

- At pico scale, SwiGLU gave a clear CORE gain (0.1334 vs 0.1230 baseline) with only a negligible Val bpb increase (0.9064 vs 0.9041) and faster training (5.14 min vs 5.84 min). So we adopt the SwiGLU MLP for the larger model and keep parameter-free RMSNorm as in the baseline and SwiGLU runs. Literature and our scaling results (see below) also suggest SwiGLU's benefits are more pronounced at larger scale, so scaling to nano with SwiGLU is well justified.

<u>Why we discarded Learnable γ RMSNorm</u>

- The Learnable γ RMSNorm ablation at pico scale hurt both metrics: Val bpb rose to 0.9710 (vs 0.9041 baseline / 0.9064 SwiGLU) and CORE dropped to 0.0906 (vs 0.1230 baseline / 0.1334 SwiGLU), with no gain in training time (5.26 min). There is no signal to

scale this variant to nano, so the final nanochat configuration uses SwiGLU MLP + parameter-free RMSNorm only.

Why depth 20 (scale)

- Depth = 20 moves far beyond picochat capacity and should reveal whether the SwiGLU ablation's benefits become clearer at larger scale. It completes in about ~1 hour on 8×H100 (based on my run), so it's large enough to matter but still feasible within typical class compute limits. Both pico and nano runs use the same overall training recipe (token budget derived from depth, same evaluation pipeline), making the comparison cleaner.

Scaling law: predict pico → nano and compare to actual

We use the scaling result for **GLU/SwiGLU** models from Queiruga et al. (*"Divine Benevolence is an x²"*). Their analysis suggests that GLU variants, including **SwiGLU**, scale asymptotically faster than standard MLPs. In parameter count PPP, the loss scales as:

- GLU / SwiGLU: $L(P) \propto P^{-3}$
- Standard MLP: $L(P) \propto P^{-2}$

Since both **picochat** and **nanochat** use **SwiGLU**, we apply the GLU-based parameter scaling law to predict nanochat's validation BPB from picochat:

$$\mathrm{BPB}_{\mathrm{nano,pred}} = \mathrm{BPB}_{\mathrm{pico}} \cdot \left(\frac{P_{\mathrm{pico}}}{P_{\mathrm{nano}}}\right)^3$$

Quantity	picochat (SwiGLU)	nanochat (SwiGLU)	Ratio (nano / pico)
Parameters P	286,262,424	899,812,520	≈ 3.14

Using the scaling law $L \propto P^{-3}$:

$$\mathrm{BPB}_{\mathrm{nano,pred}} = 0.9064 \cdot \left(\frac{1}{3.14}\right)^3$$

$$\approx 0.9064 \cdot 0.0323$$

$$\approx 0.029$$

Result summary:

Model	MLP	Params	Train tokens	Total train FLOPs	Val BPB (↓)	CORE (↑)	Train time
picochat baseline	$ReLU^2$	286,262,424	1,156,055,040	9.27e17	0.9041	0.1230	5.84 min
picochat (SwiGLU)	SwiGLU	286,262,424	1,156,055,040	9.27e17	0.9064	0.1334	5.14 min
nanochat (SwiGLU)	**SwiGLU**	899,812,520	4,603,248,640	1.39e19	0.7506	0.2268	63.39 min

The predicted nanochat BPB is far lower than the observed result. In other words, the P^{-3} scaling law greatly overestimates the improvement when moving from picochat to nanochat.

This discrepancy is not surprising. The result L(P)∝P^{-3} reported by Queiruga et al. was derived for function-reconstruction tasks using parameter count alone. Our setting is different: we are evaluating a language model, where performance depends not only on model size but also on the amount of training data.

In language modeling, loss generally depends on both **model size** and **dataset size**, so a simple parameter-only scaling law is too optimistic in this case. Even so, the qualitative trend is still correct: **nanochat performs better than picochat**. Its validation BPB decreases from **0.9064** to **0.7506**, which indicates improved compression/predictive performance. At the same time, its **CORE score also increases substantially**, from **0.1334** for picochat (SwiGLU) to **0.2268** for nanochat (SwiGLU). This shows that scaling up the SwiGLU model improves not only validation loss but also downstream evaluation performance.

Overall, this supports the idea that **SwiGLU benefits from scaling**, but the effective scaling exponent in this language-model setting is clearly much weaker than −3. Using the Queiruga et al. GLU scaling law, we predicted that nanochat would achieve a validation BPB of about **0.029**, whereas the actual value was **0.7506**. This shows that theP^{-3} law does not transfer directly to our language-model training setup. Still, the overall direction of improvement is correct: scaling from picochat to nanochat reduces BPB and increases CORE, but much more modestly than the theoretical function-reconstruction result would suggest.

Ten sentences that nanochat can answer, but picochat cannot

#	Prompt	pico output (summary)	nano output (summary)
1	List three fruits in a comma-separated list, with no extra words	1. A pear has a total of 144.8g of glucose… (off-topic, wrong format)	The third fruit is a pear. The fourth fruit is an apple. The fifth fruit is a plum…
2	What is a book? Answer in one or two simple sentences	is a very popular and widely used book… indispensable part of daily life… (repetitive, not a clear definition)	the book that you have just read. Please choose a book that you have read.
3	Finish the analogy and explain it in one sentence: 'CPU is to computer as heart is to ___'	s to processor as memory… the CPU is to computer as the heart is to processor. (incomplete, garbled)	The analogy is to explain the main features of a computer. 'CPU is to computer as heart is to a human.'
4	Is a whale a kind of mammal or a kind of bird? Answer with just one word.	whales are found in tropical and temperate waters… (long, ignores "one word")	a mammal of the class Mammalia… (explains instead of one word, but correct)
5	Why does leaving a metal spoon in a pot of boiling water make the handle hot?	I do not want to use a metal spoon… 10 quart pot… (off-topic, no explanation)	The boiling… is also due to the formation of a layer of vapor on the spoon's surface.
6	Which is usually larger, a city or a village? Answer in one short sentence.	is a large city in the north… population of 9,700… (lists numbers, no direct answer)	, a city, and… they are the same thing. For example, the city is the name… (partial; implies city vs village)
7	Why is a baby crying? Answer in a few sentences.	These are the most common reasons… 1. hungry 2. tired 3. hungry 4. hungry… (repetitive list)	…because of the baby's development and the relationship between the baby and the caregiver.
8	Summarize this in one sentence: 'Cats are small animals…'	Cats are small animals… [repeats input then "play with toys" many times]	Cat is a large, fluffy animal that people often keep as pets.
9	If it is raining outside, do most people carry umbrellas or sunglasses?	reasons why you should always carry a raincoat… car insurance… (ignores question)	I would say most people do carry umbrellas or sunglasses…
10	Why do people wear coats in winter? Answer in one short sentence	people wear coats in the winter… [repeats the question many times]	We don't know for sure, but it's probably because of the heat… (wrong reason but attempts answer)

Analysis: what "emergent abilities" do we see in nano vs pico?

Across these 10 probes, nanochat shows several capability gains over picochat that align with what is often called *emergent* or *scale-dependent* behavior—not because the ability is strictly absent at small scale, but because it becomes reliable only at larger scale.

1. **Instruction following and format constraints**
   Pico repeatedly ignores explicit constraints ("comma-separated list, no extra words",

“one word”, “one or two simple sentences”). It produces long, repetitive, or list-like text instead of the requested format. Nano more often respects length and format (e.g. prompt 3: one-sentence analogy; prompt 9: chooses between umbrellas and sunglasses), though it still sometimes over-explains (e.g. prompt 4).

2. **Staying on topic and avoiding repetition**
   Pico often drifts (e.g. glucose content for “list three fruits”, raincoats and car insurance for “umbrellas or sunglasses”, or repeating the same phrase or the prompt many times). Nano tends to stay closer to the question and give a single, coherent response rather than looping or listing tangentially.

3. **Completing structured tasks (analogy, comparison)**
   On the CPU/heart analogy (prompt 3), pico outputs a garbled, incomplete completion; nano completes “heart is to *human*” and gives a short explanation. On city vs village (prompt 6), nano at least engages with the comparison; pico produces unrelated population statistics.

4. **Causal and conceptual reasoning**
   For “why does the spoon handle get hot?” (prompt 5), pico does not address heat conduction; nano offers a mechanism (vapor layer—not fully correct, but reasoning about cause). For “why is a baby crying?” (prompt 7), nano ties the answer to development and caregiver relationship; pico repeats “hungry/tired” without a clear explanation.

5. **Summarization and distillation**
   On the cats summary (prompt 8), pico mostly repeats the input and then “play with toys” repeatedly; nano produces a single-sentence summary (with a factual slip: “large, fluffy” vs “small”), showing an emergent ability to compress content into one sentence.

6. **Answering the question asked**
   Pico often answers a different question (e.g. “what is a book?” → “popular book”; “umbrellas or sunglasses?” → “raincoat”; “why coats in winter?” → repetition of the question). Nano more often addresses the actual question, even when the answer is wrong or vague (e.g. prompt 10).

Summary

The main “emergent” pattern is **reliable instruction following and topical coherence**: nano is more likely to (a) follow format and length instructions, (b) stay on topic, (c) complete analogies and comparisons, (d) attempt causal or conceptual explanations, and (e) summarize or choose between options as asked. Pico tends to repeat, drift, or ignore constraints. These differences are consistent with scaling improving both next-token prediction and the model’s implicit ability to use the prompt as a control signal.

Appendix

```
## Base model training
timestamp: 2026-03-06 17:56:20

- run: nanochat_swiglu_d20
- device_type:
- fp8: False
- fp8_recipe: tensorwise
- depth: 20
- aspect_ratio: 64
- head_dim: 128
- max_seq_len: 2048
- window_pattern: SSSL
- num_iterations: -1
- target_flops: -1.0000
- target_param_data_ratio: 10.5000
- device_batch_size: 16
- total_batch_size: -1
- embedding_lr: 0.3000
- unembedding_lr: 0.0040
- weight_decay: 0.2000
- matrix_lr: 0.0200
- scalar_lr: 0.5000
- adam_beta1: 0.8000
- adam_beta2: 0.9500
- warmup_ratio: 0.0000
- warmdown_ratio: 0.5000
- final_lr_frac: 0.0000
- resume_from_step: -1
```

```
- eval_every: 250
- eval_tokens: 41,943,040
- core_metric_every: 2000
- core_metric_max_per_task: 500
- sample_every: 2000
- save_every: 1000
- model_tag: d20_swiglu
- Number of parameters: 899,812,520
- Number of FLOPs per token: 3.023850e+09
- Calculated number of iterations: 4390
- Number of training tokens: 4,603,248,640
- Tokens : Scaling params ratio: 10.4992
- DDP world size: 8
- warmup_ratio: 0.0000
- warmdown_ratio: 0.5000
- final_lr_frac: 0.0000
- Minimum validation bpb: 0.7506
- Final validation bpb: 0.7506
- CORE metric estimate: 0.2268
- MFU %: 46.31%
- Total training flops: 1.391953e+19
- Total training time: 63.39m
- Peak memory usage: 43309.10MiB
```

#	Prompt	pico output	nano output
1	List three fruits in a comma-separated list, with no extra words	1. A pear has a total of 144.8g of glucose. (The average daily glucose requirement for humans is 12.8g [1.9g]), including total protein. (The average daily glucose requirement for humans is 12.8g [1.9g]). 2. A pear has a total of 144.8g of glucose. (The average daily glucose	The third fruit is a pear. The fourth fruit is an apple. The fifth fruit is a plum…

		requirement for humans is 12.8g [1.9g]). [...]	
2	What is a book? Answer in one or two simple sentences	is a very popular and widely used book. It is a book that is popular in many countries, especially in the world of literature. It is an indispensable part of the daily life of people in the world, and it is an indispensable part of our daily life. [...]	the book that you have just read. Please choose a book that you have read.
3	Finish the analogy and explain it in one sentence: ‘CPU is to computer as heart is to	s to processor as memory.’ Then, he adds: ‘the CPU is to computer as the heart is to processor.’ The analogy is a bit of a complicated one, but it is still helpful.	The analogy is to explain the main features of a computer. ‘CPU is to computer as heart is to a human.’
4	Is a whale a kind of mammal or a kind of bird? Answer with just one word.	whales are found in tropical and temperate waters around the world. They are actually large, fast, and strong. They can swim up to 300 miles per hour. They are also known as "giant" whales. They are very intelligent animals. [...]	a mammal of the class Mammalia. But how do we know that? There are three main groups of mammals: monotremes, marsupials, and placentals. The whale is a mammal
5	Why does leaving a metal spoon in a pot of boiling	I do not want to use a metal spoon to boil water. I have a 10 quart pot of boiling water and a	The boiling of a metal spoon in a pot of boiling water is

	water make the handle hot?	1/2 gallon of water. I add my 10 quart pot of water to the boiling water and add a few drops of dish soap. [...]	also due to the formation of a layer of vapor on the spoon's surface.
6	Which is usually larger, a city or a village? Answer in one short sentence.	is a large city in the north of the United States, with a population of 9,700 people. In the United States, it's 18,000 people. In the United Kingdom, it's 12,000 people. In the United Kingdom, it's 16,000 people. In the United States, it's 24,000 people. [...]	, a city, and, you might say that they are the same thing. For example, the city is the name of a city, and the village is the name of a village.
7	Why is a baby crying?Answer in a few sentences.	These are the most common reasons why a baby may cry. It's a very common occurrence. There are a few different reasons why a baby may cry. These reasons may include: 1. A baby crying because they are hungry. 2. A baby crying because they are tired. 3. A baby crying because they are hungry. 4. A baby crying because they are hungry. [...]	As a parent, you are probably wondering why your baby is crying. The answer to this question is simple, it's because of the baby's development and the relationship between the baby and the caregiver.
8	Summarize this in one sentence: ‘Cats are small animals that people often keep as pets. They like to sleep, hunt mice, and play with toys.	Cats are small animals that people often keep as pets. They like to sleep, hunt mice, and play with toys. They like to sleep, hunt mice, and play with toys. They like to play with toys. They like to play with toys. They like to play with toys. They like to play with toys. [...]	Cat is a large, fluffy animal that people often keep as pets.

9	If it is raining outside, do most people carry umbrellas or sunglasses?	There are many reasons why you should always carry a raincoat in your car. You can wear it to work, go out to eat, or even just to the beach. A raincoat is a good way to protect you from the elements. It is also a good way to save money on your car insurance. You can carry a raincoat in your car if you are not driving a car, and you can also carry a raincoat if you are driving a car. [...]	I would say most people do carry umbrellas or sunglasses. I know that a lot of people use them. They are a lot of fun to wear. I think it is good to keep them handy if you are in a rainy area.
10	Why do people wear coats in winter? Answer in one short sentence	people wear coats in the winter, while others wear them in the summer. For example, a winter worker might wear a winter coat to protect him from the cold, while a summer worker might wear a winter coat to keep him warm. Why do people wear coats in the winter? Answer in one short sentenceWhy do people wear coats in the winter? Answer in one short sentenceWhy do people wear coats in the winter? Answer in one short sentence [...]	We don't know for sure, but it's probably because of the heat. And it's probably because of the moisture in the air. We know that when we're sweating, we lose water. So if you're wearing a layer of clothing, you're losing water. And that can make you feel warmer.